

Building Flask Applications (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-flask-application-structure>  <https://github.com/learn-co-curriculum/python-p4-flask-application-structure/issues/new>

Learning Goals

- Describe the components of a web application framework.
 - Build and run a Flask application on your computer.
 - Manipulate and test the structure of a request object.
-

Key Vocab

- **Web Framework:** software that is designed to support the development of web applications. Web frameworks provide built-in tools for generating web servers, turning Python objects into HTML, and more.
 - **Extension:** a package or module that adds functionality to a Flask application that it does not have by default.
 - **Request:** an attempt by one machine to contact another over the internet.
 - **Client:** an application or machine that accesses services being provided by a server through the internet.
 - **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
 - **Web Server Gateway Interface (WSGI):** an interface between web servers and applications.
 - **Template Engine:** software that takes in strings with tokenized values, replacing the tokens with their values as output in a web browser.
-

Introduction

It's finally time to create our first Flask application. We'll start this lesson with your standard "text output in the browser" application, but we'll also extend a bit further to explore **routing**, creating Flask development servers, and debugging in the browser.

Initializing an Application

Flask is a web framework, but it is first and foremost a *Python* framework. To start any Flask application, you need to create an instance of the `Flask` class. Your web server- whether you run it from Werkzeug, Flask, or another library entirely- interacts with this `Flask` instance using WSGI.

Before we start, make sure to enter your virtual environment with `pipenv install && pipenv shell`.

Open `server/app.py` and enter the following code:

```
from flask import Flask

app = Flask(__name__)
```

The `Flask` class constructor only requires the name of the primary module or package to be interpreted as the application. Flask uses this to figure out where the application is and where its important files will be. As some of these will not be `.py` files and might not have any `.py` files in their directory, Flask needs to set up an application structure that allows it to see everything.

For the purposes of our curriculum, this argument will always be `__name__`, which refers to the name of the current module.

► What is the value of `__name__` when we run `python server/app.py`?

Routing Views

Routing

When clients send requests to our application's server, they are forwarded to our Flask application instance. This instance will receive requests for many different resources, located at different Uniform Resource Locations (URLs). To map these URLs to Python functions, we need to define **routes**.

Routing is the association of URLs and the code that should execute when a request comes in for that URL. Routing isn't just a Python concept- JavaScript, Java, Ruby, and even newer languages like Rust and Go use routes to direct requests to the appropriate backend code.

The easiest way to define routes with Flask is through use of the `@app.route` decorator:

```
# append to server/app.py

@app.route('/')
def index():
    return '<h1>Welcome to my page!</h1>'
```

Remember that **decorators** are functions that take functions as arguments and return them decorated with new features. `@app.route` registers the `index()` function with the Flask application instance `app`. The `@app.route()` decorator is an instance method that modifies `app`, creating a rule that requests for the base URL (`/`) should show our index: a page with a header that says "Welcome to my app!"

Views

We call functions that map to URLs **views**. This is easy to remember- everything you *view* in your application is generated by a **view** (though there are certainly many invisible things going on in views as well).

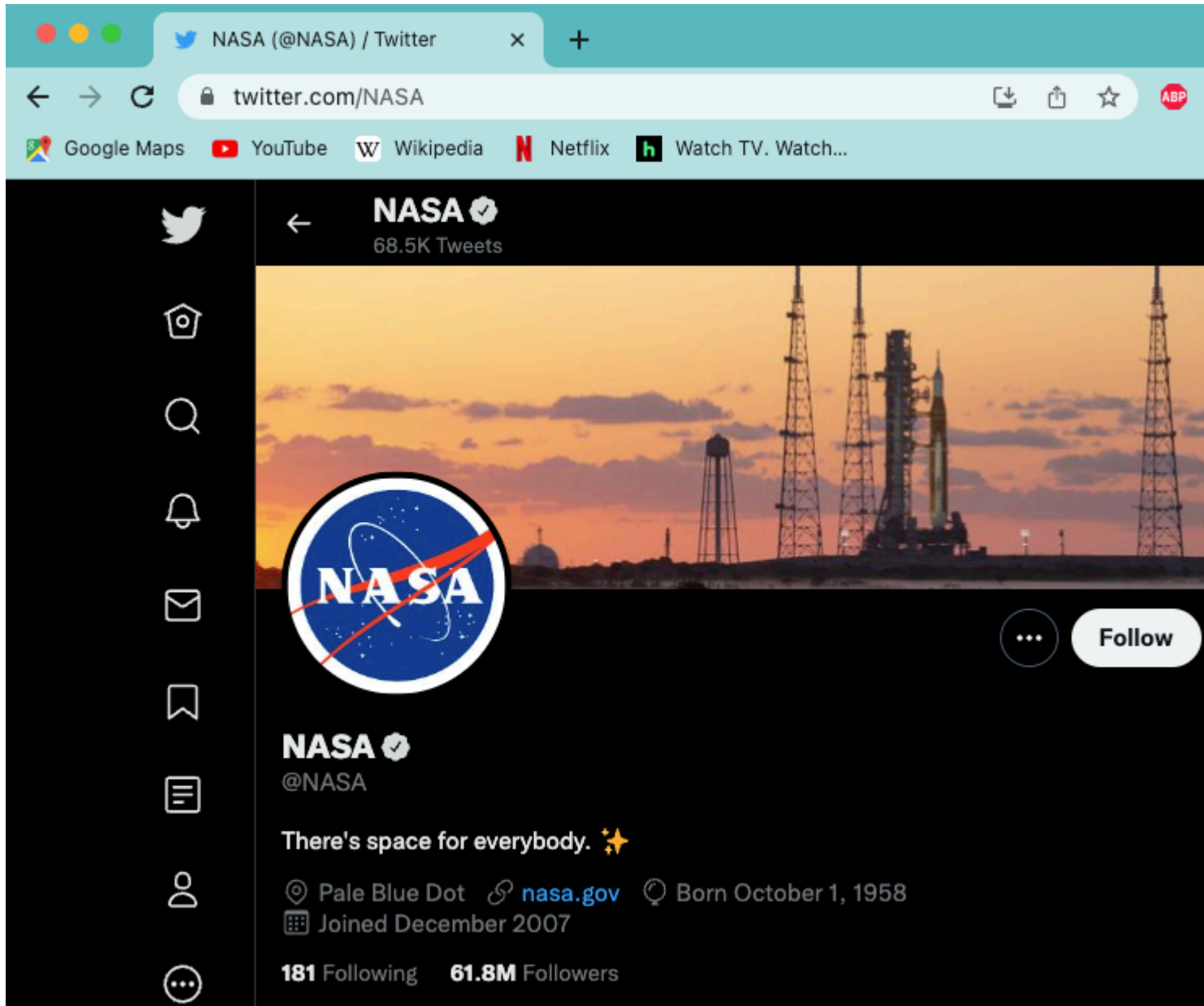
A view returns the response that the client delivers to the user. Our `index()` view returns a simple string of HTML code, but we will see throughout Phase 4 that views can also contain forms, code to ensure cybersecurity, and much more.

► Which line of code tells Flask to show the returned data from `index()` in the web browser?

Variable Routes

Navigate to your favorite social media site and take a look at the URL. The base will represent the index or homepage for the application.

Navigate to a user profile and take another look at the URL:



"twitter.com" is clearly a fixed portion of the URL- it's everywhere! There are other pieces, though, that it wouldn't make sense to hard-code into our application. "NASA", for instance, is the username for one out of millions of users on the site. Managing views for that many users would be impossible!

Lucky for us, Flask allows us to parameterize different parts of our routes. When we interpolate these into strings or use them to retrieve records from a database, we can create flexible, dynamic applications:

```
# append to server/app.py

@app.route('/<username>')
def user(username):
    return f'<h1>Profile for {username}</h1>'
```

Anything included in the route passed to the `app.route` decorator with angle brackets `<>` surrounding it will be passed to the decorated function as a parameter. We can make sure that the username is a valid `string`, `int`, `float`, or `path` (string with slashes) by specifying this in the route:

```
# modify user() in server/app.py

@app.route('/<string:username>')
def user(username):
    return f'<h1>Profile for {username}</h1>'
```

► *Flask has to parse "string" and "username" from the route. How can you use Python to remove brackets and get parameters out of a string?*

Now we're ready to run our application.

The Flask Development Web Server

In the last lesson, we ran our development server through Werkzeug. We could do that here, but we would have to reconfigure it to work specifically with our Flask application. We'll put in the work there when we have a complete application to show the whole world. For now,

there's an easier way: `flask run` .

`flask run` is a command run from the console that looks for the name of the Python module with our Flask application instance. To run the application that we created in this lesson, we need to run two commands inside of our `pipenv` virtual environment, from the `server/` directory:

```
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
$ flask run
# => * Serving Flask app 'app.py'
# => * Debug mode: off
# => WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
# => * Running on http://127.0.0.1:5555
# => Press CTRL+C to quit
```

NOTE: Flask's default port is 5000, which conflicts with AirPlay on MacOS. we manually set the port to 5555 here to avoid this conflict.

Navigate to `http://127.0.0.1:5555` and you should see the index for our application:

localhost:5555

Welcome to my page!

NOTE: "localhost" is the plain-English version of 127.0.0.1 on most machines. You will often see the two used interchangeably.

Add a username to the URL. Now you should see something like this:

  localhost:5555/Mr-User

Profile for Mr-User

We can also run a development server through treating our application module as a script with the `app.run()` method:

```
# append to server/app.py

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Run the script and you should see that we're running the same server as before:

```
$ python app.py
# => * Serving Flask app 'app'
# => * Debug mode: off
```

```
# => WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
# => * Running on http://127.0.0.1:5555
```

`debug=True` provides us many benefits for development, but the nicest of these is that the server will *automatically restart* whenever we change `app.py` !

There are pros and cons to each approach. Configuring the `FLASK_APP` and `FLASK_RUN_PORT` environment variables allows us to use tools like the Flask shell, but it's easy to forget about environment variables since they don't show up in your project directory.

Running from a script isn't quite as Flasky, but it keeps all of our configuration in sight. We also still have access to these Flask tools as written, because `flask run` and `flask shell` look for `app.py` by default!

Conclusion

You've just built your first Flask web application! It's very simple, but you'll use the Flask class and its decorator methods many times throughout Phase 4. Next, we'll get some more practice with routing and views.

Solution Code

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def index():
    return '<h1>Welcome to my page!</h1>'

@app.route('/<string:username>')
def user(username):
```

```
return f'<h1>Profile for {username}</h1>'

if __name__ == '__main__':
    app.run(port=5555, debug=True)
```

Resources

- [A Minimal Application - Flask](https://flask.palletsprojects.com/en/2.2.x/quickstart/#a-minimal-application) ➞ (https://flask.palletsprojects.com/en/2.2.x/quickstart/#a-minimal-application)
- [Routing - Flask](https://flask.palletsprojects.com/en/2.2.x/quickstart/#routing) ➞ (https://flask.palletsprojects.com/en/2.2.x/quickstart/#routing)
- [Chapter 2. Basic Application Structure - Flask Web Development, 2nd Edition](https://learning.oreilly.com/library/view/flask-web-development/9781491991725/ch02.html#idm140583868985008) ➞ (https://learning.oreilly.com/library/view/flask-web-development/9781491991725/ch02.html#idm140583868985008)

This tool needs to be loaded in a new browser window

Load Building Flask Applications (CodeGrade) in a new window